

EXECUTIVE SUMMARY

Sendline: from a local operator tool to a hosted, multi-user product

Sendline lets an operator load a curated list of people, write one personalized message, optionally attach a file, and launch a worker that opens Chrome and sends those LinkedIn messages one by one — with a live log and the ability to stop at any time. Today that workflow runs on a single Windows PC: one operator, one LinkedIn session, credentials and customer lists stored in plain files on disk. Hosting that design on a public URL would expose every user’s LinkedIn login and outreach data. This note summarises how Sendline becomes a multi-user service without that leak.

The destination product

Each person logs into **Sendline** in the browser with their own account. Each person connects **their own LinkedIn login** — other operators, and admins, cannot see or use it. Customer lists, attachments, and Chrome sessions are private to that user. When they click Launch, Chrome runs in an isolated cloud environment for that user only, so several operators can work at once without sharing sessions, passwords, or data. This is a new product shape, not a small patch on the current local app.

Two kinds of account (keep them separate)

- **Sendline user** — how they sign into our product. Store only a password hash. Admins invite, enable, or disable people, and can reset the Sendline password.
- **LinkedIn login** — belongs to that one Sendline user. Nobody else lists it, decrypts it, or launches a worker with it. Admins do not impersonate LinkedIn.

How sensitive data is held

Plain JSON on disk is not acceptable. LinkedIn passwords are the highest-risk item: even an encrypted database is dangerous if a server breach also yields the encryption keys. Preferred approach is to keep an encrypted LinkedIn *session* (cookies) and ask the user to re-enter the password only when that session dies. If a password must be stored in v1, it is envelope-encrypted with a cloud KMS, decrypted only in worker memory for seconds, never returned to the UI, and never written to logs.

Data	Where it lives	Rule
Sendline password	Users table	Hash only (Argon2id)
LinkedIn username / password	Secrets table by user	Encrypt at rest; UI never shows the password after save
Customer names / lists	Campaigns & people tables	Every row tagged to one user; database row-level security
Attachments	Private object storage	Per-user folder; signed URLs; not a public bucket
Chrome cookies / profile	Encrypted volume per user	One profile per person; wiped on account deletion

Isolated browsers in the cloud

One shared Chrome on one server will mix cookies and sessions, and the current worker kills every Chrome process on the machine. The rule is **one isolated browser environment per active user**. “Isolated” means User A’s Chrome, cookies, debug port, and files cannot be seen or killed by User B. “Active” means the environment exists for a run (and maybe a short warm window), not a 24/7 virtual machine for every signup. At most one campaign per user at a time; different users may run in parallel.

Recommended v1: a Linux container per Launch, with an encrypted disk folder unique to that user mounted as Chrome’s profile. Selenium talks to Chrome only inside that container — the debug port is never published on the internet. A small dedicated VM per active user is the later upgrade if LinkedIn blocks shared datacentre IP addresses and a stable egress IP is required. Idle users do not hold a running machine.

Risks that must stay on the table

Automating LinkedIn messaging can violate LinkedIn’s terms and can lead to account restrictions. Running that automation from cloud IPs is more likely to look like a bot farm than a home PC. This is a product, legal, and operational risk — not only an engineering task. Stakeholders should accept it in writing before a public launch.

What we will not do

- Put the existing local Flask app on a public server and call it multi-user.
- Let several operators share one LinkedIn inbox.
- Return or log LinkedIn passwords.
- Run everyone’s Chrome in one shared browser profile.

Build in four stages

1	Accounts & tenancy	Sendline login, sessions, and every campaign, list, and file tagged to one user.
2	Secret vault	LinkedIn credentials out of JSON files; UI shows “connected” not the password.
3	Per-user cloud browsers	Job queue, isolated containers, encrypted profiles, private live logs.
4	Admin console	Invite and disable users; usage and audit of launches — never password viewing.

Bottom line: Sendline can be hosted for many operators, but only if each person has a Sendline identity, their own encrypted LinkedIn connection, isolated customer data, and a private Chrome environment for each run. Shipping the current single-PC design to the public internet is not a shortcut — it is a data-loss event.

Source: Sendline multi-user hosting plan (accounts, secret isolation, per-user cloud Chrome). This summary is for internal decision-makers; it is not a LinkedIn partnership or an official API product.